

Reviewer Pack — Minimal Order of Formal Languages and DPLL Proof Length

Entry f08d7a1d940c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 19:49:40 UTC

Minimal Order of Formal Languages and DPLL Proof Length

Entry ID: f08d7a1d940c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 19:49:40 UTC

1. Conjecture

Field A (mathematical branch): Formal Language Theory (Grammar Complexity) **Field B** (complexity object): DPLL Search Trees

Statement:

For every Boolean satisfiability instance φ , the minimal order of a regular grammar generating the language $L(\varphi)$ is linearly correlated with its DPLL proof length, such that $OLG_L(\varphi) = \Theta(d(\varphi))$, where OLG_L denotes the order of the smallest regular grammar and $d(\varphi)$ is the DPLL proof length.

Rationale (proposer's reasoning):

Formal language theory has provided insights into the structure of computational problems, particularly in terms of grammatical complexity. A connection between this and the DPLL proof length could expose a fundamental relationship between syntax and semantics in the context of SAT solving.

Taxonomy category: GRAMMAR_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8307958a63ced91a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if Pearson correlation coefficient of $OLG_L(\varphi)$ and $d(\varphi)$ exceeds 0.8 for all seeds, where $OLG_L(\varphi)$ is the order of the smallest regular grammar and $d(\varphi)$ is the DPLL proof length, and falsified if this condition is not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order formal language" AND "DPLL proof length" - "grammar complexity" AND "DPLL search trees" - "order of regular grammar" related TO "Boolean satisfiability instance"

Top relevant hits considered: - [http://arxiv.org/abs/cs/0608100v1] Similarity of Semantic Relations - [http://arxiv.org/abs/0810.1207v1] A Layered Grammar Model: Using Tree-Adjoining Grammars to Build a Common Syntactic Kernel for Related Dialects - [http://arxiv.org/abs/1310.8154v3] Characteristic cohomology of the infinitesimal period relation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_formula(n):
    if n == 1:
        return 'T' if random.choice([True, False]) else 'F'
    elif n == 2:
        op = random.choice(['&', '|'])
        left = generate_formula(1)
        right = generate_formula(1)
        return f"({left} {op} {right})"
    else:
        op = random.choice(['&', '|'])
        left = generate_formula(n // 2)
        right = generate_formula(n - n // 2)
        return f"({left} {op} {right})"

def dpll(formula):
    if formula == 'T':
        return []
    elif formula == 'F':
        return None
    else:
        var, op, subformula = formula[1:-1].split()
        if op == '&':
            left = dpll(subformula)
            right = dpll(subformula.replace(var, 'F'))
            if left is not None and right is not None:
```

```

        return left + right
    elif left is not None:
        return left
    else:
        return right
elif op == '|':
    left = dpll(subformula)
    right = dpll(subformula.replace(var, 'T'))
    if left is not None or right is not None:
        return left or right
    else:
        return None

def regular_grammar_order(formula):
    if formula == 'T' or formula == 'F':
        return 1
    elif formula[0] == '(' and formula[-1] == ')':
        var, op, subformula = formula[1:-1].split()
        if op == '&':
            left = regular_grammar_order(subformula)
            right = regular_grammar_order(subformula.replace(var, 'F'))
            return max(left, right) + 1
        elif op == '|':
            left = regular_grammar_order(subformula)
            right = regular_grammar_order(subformula.replace(var, 'T'))
            return min(left, right) + 1

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        formula = generate_formula(n)
        dpll_length = dpll(formula)
        if dpll_length is None:
            continue
        order = regular_grammar_order(formula)
        results.append((order, dpll_length))

    if len(results) < 30:
        return {
            "metric_name": "Pearson correlation coefficient",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    order_values = [r[0] for r in results]
    dpll_length_values = [r[1] for r in results]

    n = len(order_values)
    mean_order = sum(order_values) / n
    mean_dpll_length = sum(dpll_length_values) / n

```

```

covariance = sum((order_values[i] - mean_order) * (dpll_length_values[i] - mean_dpll_length) for i in range(n))
variance_order = sum((order_values[i] - mean_order) ** 2 for i in range(n)) / n
variance_dpll_length = sum((dpll_length_values[i] - mean_dpll_length) ** 2 for i in range(n))

pearson_correlation = covariance / (math.sqrt(variance_order) * math.sqrt(variance_dpll_length))

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": pearson_correlation,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": pearson_correlation >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_24f6cc9d.py", line 121, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_24f6cc9d.py", line 77, in run_trial
    dpll_length = dpll(formula)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_24f6cc9d.py", line 38, in dpll
    var, op, subformula = formula[1:-1].split()
                          ~~~~~^

```

ValueError: too many values to unpack (expected 3)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the Pearson correlation coefficient could not be calculated to verify the conjecture. | next: Investigate and fix the crash in the test code to allow for the calculation of the Pearson correlation coefficient and verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19237
2	preregistration	ollama_remote	glm4:latest	0	0	11107
3	novelty	ollama_remote	glm4:latest	0	0	8293
4	novelty	ollama_remote	glm4:latest	0	0	20386
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17503
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16568
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	197544
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	104581
9	judge	ollama_remote	glm4:latest	0	0	33765

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 428985 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f08d7a1d940c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f08d7a1d940c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f08d7a1d940c.tar.gz` (if generated)